技术可行性评估

AgentScope 引入可行性与改动成本分析

自研 Go 引擎 vs 阿里 AgentScope 2.0，改动成本与独立部署可行性

项目	睿乐大脑
版本	v1.0
日期	2026-10-09
结论	旁路试点可行，整体替换不建议

4	24	34
硬接缝数量	纯库编排人日	含Team人日

42
替换专家流人日

AgentScope 引入可行性与改动成本分析

项	内容
评估对象	阿里通义实验室开源的 **AgentScope 2.0**（Python 多智能体框架 + Agent Service）
评估范围	① 在本项目中使用 AgentScope 的改动成本；② 以独立应用服务方式部署是否可行
被评估方	睿乐大脑（`/Users/jamgogh/Desktop/agent/ruile`），Go 自研 ReAct Agent 引擎
结论口径	所有结论均附 `文件:行号` 证据；成本为**人日估算**（1 人日 = 8h），不含需求评审与外部依赖等待
版本	v1.0 · 2026-10-09

0. 结论摘要

0.1 一句话结论

技术上完全可行，但「替换现有 Agent 引擎」是一条高成本路线（约 100 人日起），不建议；「把 AgentScope 作为旁路独立服务、只承接多 Agent 编排这类增量场景」成本可控（约 24 人日起），建议先做。

单独部署一个应用服务——**可行，且本项目已有成熟范式**（docreader、mcp-server 都是 Python 独立服务，已进 compose 与 helm）。真正的门槛不在部署，而在**模型凭证、工具能力、计费、事件契约**这四道接缝。

至于**「有没有价值」——有，但价值不在框架，在三项能力**：可编排多 Agent（唯一强绑定框架的增量）、隔离沙箱、长期记忆；其余八项要么与项目重叠、要么是负债。所以结论是：**作为「设计参考 + 旁路引擎」有价值，作为「平台底座 / 主引擎」价值为负**。详见 §3.9。

但在动手引入任何外部框架之前，还有一条成本更低的路径被遗漏了：本项目是 WeKnora 的定制分支（当前 `VERSION` = 0.7.0，上游已到 0.8.2），而上游 0.8.0 已内建**沙箱运行时（Docker / E2B / Cube）、跨会话长期记忆、租户级 Skill Catalog、消息产物库、token 用量归属**——也就是说，**AgentScope 的增量里除「多 Agent 编排」之外，上游全部已有**，移植成本约 8~14 人日。**推荐顺序因此修正为：先移植上游 0.8，再用 AgentScope 旁路补多 Agent 缺口。** 详见 §3.11。

0.2 三档方案对比

方案	做法	改动量	人日估算	建议
A. 旁路试点	AgentScope 独立进程，只服务新场景（多 Agent 编排类），现有 `/agent-chat` 主链路不动	新增为主，几乎不改存量	≈ 24	✅ **推荐先做**
B. 引擎替换	用 AgentScope 替换 Go ReAct 引擎，Go 侧退化为网关与平台层	19.6k 行 Go Agent 代码失效 + 四道接缝全改	≈ 99	⚠️ 除非多 Agent 成为刚需
C. 平台级采用	直接用 AgentScope Agent Service 承压（含其 Web UI / RAG / IM / 调度）	与本项目产品层大面积重叠，需做能力取舍	≈ 190~215	❌ 不建议

> 若只取「多 Agent 引擎」而不引入整个框架，成本可进一步分档到 20 / 24 / 34 / 42 人日——详见 **§3.6**（含一个关键认知纠正：2.0 的 Leader-Worker 动态派生**不是纯库能力**）。若只取「智能体本体」（不用"智能体即服务"），那是性价比最差的一档——详见 **§3.7**。**投产比测算（换框架能否打正）见 §3.8**：换整框架打不正，换引擎取决于一个未验证的假设，旁路多 Agent 最容易打正。**「是否有价值」的综合判断见 §3.9**：有价值，但价值在三项能力而非框架本身。**反向问题「不引入框架、自研复刻这些能力的复杂度」见 §3.10**：约七成能力项目已有等效物，纯人日自研更贵（包 A 32~40、包 B 70~88），但买到的是零外部依赖与零双栈；最优解是**中间件自建、编排调度引入**。**上游框架核查见 §3.11**：睿乐是 WeKnora 0.7.0 分支、上游已到 0.8.2，而**沙箱 / 长期记忆 / Skill Catalog / 产物库 / 会话控制上游**

> **都已经有**（8~14 人日可移植），**只有多 Agent 编排上游也没有**——因此推荐路径应修正为：**先移植上游 0.8，再用 AgentScope 旁路补多 Agent 缺口**。

0.3 关键判断（三条）

1. **AgentScope 的增量价值集中在三处**：原生多 Agent 编排（TeamPipeline / Leader-Worker / A2A / SOP）、隔离沙箱与 workspace、长期记忆后端（ReMe / Mem0）。这三处恰好是本项目当前最弱的地方——专家工作流是一段**硬编码的五阶段流水线**（`internal/application/service/agent_run_expert_workflow.go:140`），沙箱默认关闭（`internal/application/service/agent_service.go:359-398`）。
2. **AgentScope 与项目重叠的部分，恰恰是产品的护城河**：多租户、RAG、IM 渠道、计费、RBAC、前端契约。这部分交给框架 = 把已经做好的东西重做一遍。
3. **最大的隐性成本是「双语言 Agent 栈」**：工具、事件、提示词、上下文策略会在 Go 与 Python 各存一份，长期维护成本高于一次性迁移成本。

---

1. 事实基线

1.1 本项目现状：自研 Go ReAct 引擎

结论：本项目 Agent 能力是完整自研的，不依赖任何第三方 Agent 框架；规模约 2.6 万行（Agent 核心 + 服务/Handler 层）。

层次	位置	规模（非测试）
ReAct 引擎	`internal/agent/`	**19,653 行**（含工具）
└ 工具系统	`internal/agent/tools/`	12,320 行 / 24 个内置工具
└ 上下文压缩	`internal/agent/compaction/` + `token/`	1,358 行
服务层	`agent_service.go` / `session_agent_qa.go` / `agent_run.go` / `agent_run_expert_workflow.go`	4,680 行
Handler 层	`qa.go` / `agent_stream_handler.go` / `stream.go`	2,503 行
事件协议	`internal/event/`	693 行

主循环：标准 ReAct 单循环。

- 入口 `internal/agent/engine.go:205` `Execute` → 主循环 `engine.go:361` `executeLoop` → 单步 `engine.go:467` `runReActIteration`
- 四阶段：think（`engine.go:551`）→ analyze（`engine.go:632`）→ act（`engine.go:674`）→ observe（`engine.go:679`）
- 引擎**跨轮无状态**，历史每轮从 DB 重建（`engine.go:31-35` 注释；重建入口 `internal/application/service/session_agent_qa.go:139` `LoadAgentHistory`）

工具系统：

- 接口定义 `internal/types/agent.go:175`（`Name` / `Description` / `Parameters` / `Execute`）
- 24 个内置工具名常量 `internal/agent/tools/definitions.go:9-34`（通用 12 + 技能 2 + Wiki 10）
- 注册 `internal/application/service/agent_service.go:445` `registerTools`，核心是 `agent_service.go:616-690` 的大 `switch`，按白名单逐项注册（`:699`）
- 工具**直接注入 Go 的 service 实例**（`agent_service.go:613-701`，如 `:622-630` 的 `NewKnowledgeSearchTool(...)`）——这是替换成本的核心来源

- 并发执行 `internal/agent/act.go:190`（`errgroup`，仅 `ParallelToolCalls` 开启时）；单工具超时 60s（`internal/agent/const.go:28`）

事件与契约：

- 引擎事件类型 `internal/event/event.go:52-57`：`thought` / `tool_call` / `tool_result` / `reflection` / `references` / `final_answer`
- 下发到前端的 `ResponseType` 共 **16 种**，`internal/types/chat.go:91-126`
- 翻译层 `internal/handler/session/agent_stream_handler.go:100-114`（订阅清单），SSE 推送 `internal/handler/session/stream.go:321`（100ms ticker 轮询）
- **两套契约并存**：会话流 `POST /api/v1/agent-chat/:session_id`（`internal/router/router.go:790`）+ 运行流 `/api/v1/agent-runs/*`（`frontend/src/api/agent-run.ts:31-99`）

消费方范围（实测）：`agent-chat` SSE 仅被 Web 端消费（`frontend/` 与其复用源 `admin/`）。移动端 `mobile/` 全部走 organize / service / knowledge 接口，小程序 `miniprogram/` 只用 `/api/v1/knowledge-chat`——**契约面比想象中窄**。

1.2 多 Agent 能力：目前只有硬编码流水线

结论：本项目没有子 Agent / Agent 间交接能力，唯一的"多角色"是一条写死的五阶段流水线。

- 工具表里不存在任何 `subagent` / `delegate` / `handoff` 类工具（对照 24 个工具名）
- `internal/application/service/agent_run_expert_workflow.go:140` `executeExpertWorkflow`：intake（`:294`）→ planning（`:446`）→ drafting（`:514`）→ quality review（`:652`）→ revising 循环（`:230-253`）→ packaging（`:697`），**步骤与顺序全硬编码**
- 每个步骤各自 `NewAgentEngine` 并 `Execute`（`:578` / `:593`），且 `AllowedTools: []string{}`（`:572`）——**纯文本生成，不挂任何工具**
- 内置 Agent 有 5 个条目（`config/builtin_agents.yaml`），但只对用户暴露 1 个（`internal/types/custom_agent.go:566` `builtinAgentIDsOrdered`）

> 这是引入 AgentScope 最站得住脚的理由：**要真正的多 Agent，现在得自己造编排层。**

1.3 AgentScope 2.0 现状

结论：AgentScope 已从「多智能体库」演进为「多智能体平台」，2.0 直接内建了服务化能力；选型必须用 2.0，不要用已归档的 `agentscope-runtime`。

维度	事实
许可	Apache-2.0（商用友好）
运行时	**Python 3.11+**
核心能力	ReAct（含实时中断/恢复）、Toolkit（Python 工具 + **MCP** + skills）、Model（OpenAI / Anthropic / Gemini / DashScope / DeepSeek / Moonshot / Volcengine / xAI / Ollama / MiniMax）、Context（自动压缩 + 工具结果卸载 + 中间件注入）、Event System（统一事件总线）、Permission & HITL、Middleware、Memory（ReMe / Mem0）、Workspace / Sandbox（local / Docker / Apple Container / Bubblewrap / E2B / OpenSandbox / Daytona / **K8s**）
模型接入	**Credential（api_key + base_url）+ Model 两层解耦**，支持任意 OpenAI 兼容端点；可显式指定 `base_url` 指向自建代理
多 Agent	`TeamPipeline`（leader 派发 worker）、内置团队工具、A2A 协议、SOP（2026-09 起）
服务化	**Agent Service**：FastAPI 后端 + 自带 Web UI；以 `create_app` 组装（`RedisStorage` + `RedisMessageBus` + `WorkspaceManager`）
多租户	「by construction」——资源按请求的 `user_id` 归属并在路由层强制；**但不含内置鉴权**，只提供 `X-User-ID` 占位头，需自行替换为 JWT/OAuth
分布式	共享状态入 Redis（存储 + 消息总线）；官方标注分布式部署为 **WIP**
观测性	内建 OpenTelemetry 追踪（`src/agentscope/middleware/_tracing/`）
归档提示	`agentscope-runtime` 仓库已转只读并即将归档，能力并入 2.0

Agent Service 自带的能力清单（注意：与本项目重叠度极高）

多租户/多会话隔离、Agent Team（Leader-Worker）、IM 渠道（飞书/Discord）、RAG 服务（多租户检索）、MCP & Skill Hub（GitHub MCP Registry / ClawHub）、资源分享（组级/组织级）、持久化（SQL & Redis）、定时调度与后台任务卸载、会话重放（SSE 断线重连补发历史）、协议适配中间件（AG-UI / A2A）。

---

2. 能力对照：AgentScope 2.0 vs 本项目

能力面	本项目	AgentScope 2.0	谁更强	是否重叠
ReAct 单 Agent 循环	自研，19.6k 行，含工具结果裁剪与溢出修复	原生，含实时中断/恢复	持平	✅ 完全重叠
工具系统	24 个内置工具，**直接绑 Go service**	Toolkit：Python 工具 + MCP + skills	AgentScope 更通用	✅ 重叠
上下文压缩	自研（reserve 16k / keep 20k）	原生自动压缩 + 工具结果卸载	持平	✅ 重叠
事件流	16 种 `ResponseType` + `agent_steps` 回写	统一事件总线 + AG-UI/A2A 适配	本项目更贴合前端	✅ 重叠但格式不同
多 Agent 编排	**仅硬编码五阶段流水线**	**TeamPipeline / Leader-Worker / A2A / SOP**	**AgentScope 明显更强**	❌ 本项目缺
沙箱隔离	自研，**默认 disabled**	local / Docker / E2B / K8s 等 8 种后端	**AgentScope 明显更强**	❌ 本项目弱
长期记忆	无独立后端	ReMe / Mem0 可切换	**AgentScope 更强**	❌ 本项目缺
多租户	Auth 中间件注入 tenant（`internal/middleware/auth.go:124`、`:251-267`）	按 `user_id` 归属，无内置鉴权	**本项目成熟得多**	✅ 重叠
RAG 检索	9 种向量库 + chat_pipeline 6,638 行	自带 RAG 服务（多租户）	**本项目成熟得多**	✅ 重叠
模型接入	24 个 provider + BYOM + AES-GCM 凭证	Credential + Model 解耦	**本项目更完整**（含凭证加密与租户隔离）	✅ 重叠
计费/额度	完整（token + 工具调用 + 账本）	**无**	**本项目独有**	❌ AgentScope 缺
IM 渠道	自研 9 个（飞书/企微/钉钉/Slack/Telegram...）	飞书/Discord/自定义	本项目更全	✅ 重叠
权限/HITL	工具审批 + MCP OAuth（`internal/agent/approval/`）	Permission & HITL + bypass 模式	持平	✅ 重叠
可观测性	Langfuse span	OpenTelemetry	需打通	✅ 重叠但后端不同
调度	有（周期整理任务）	cron 调度 + 唤醒 + 后台卸载	AgentScope 更通用	部分重叠

读法：标 ❌ 的行是引入 AgentScope 的**正当理由**（3 项）；标 ✅ 的行是引入后的**双份维护负担**（10 项）。

---

3. 问题 1：改动成本分析

3.1 四道硬接缝（成本的主要来源）

无论走哪条路线，都必须跨过这四道：

接缝① 事件协议

- 现状：引擎发 `event.EventType`（`internal/event/event.go:52-57`）→ `agent_stream_handler.go:100-114` 翻译为 16 种 `ResponseType`（`internal/types/chat.go:91-126`）→ SSE（`stream.go:321`）→ 前端 `useChatStreamHandler.ts:510+` 分支消费
- AgentScope 有自己的事件总线，并有 AG-UI / A2A 适配中间件，但**事件体结构与本项目不同**
- **必须做**：写一层协议翻译（AgentScope 事件 → 本项目 `response_type`），且要覆盖 `agent_steps` 回写（`internal/types/message.go:220`）与审批类事件（`tool_approval_required` / `mcp_oauth_required`）
- 影响：前端**理论上可零改动**（若翻译层做全），但这层翻译是新增的长期维护资产

接缝② 模型凭证

- 现状：模型配置是 DB 行（`internal/types/model.go:110-144`，表 `models`），`api_key` / `app_secret` **AES-GCM 加密落库**（`model.go:148-193`），按 `tenant_id` 隔离，平台模型 `tenant_id=0`（`model.go:37`）
- AgentScope 侧需要 `Credential(api_key, base_url)`（支持任意 OpenAI 兼容端点）
- **两条路**：
- (a) 新增内部接口把**解密后的凭证**下发给 Python 服务 —— 需配套短期令牌 + 审计，有安全面
- (b) **让 AgentScope 指向 Go 侧的 OpenAI 兼容代理** —— 推荐。凭证不离开 Go，且天然复用下面接缝④的计费
- 注意：项目现有 provider 适配器有 24 个（`internal/models/provider/provider.go:15-66`），但大部分对外就是 OpenAI 兼容协议，走 (b) 只需代理一层

接缝③ 工具能力

- 现状：24 个工具共 12,320 行 Go，且**直接绑定 Go 内部 service**（`internal/application/service/agent_service.go:613-701`）
- AgentScope Toolkit 支持 **MCP**，而本项目**已经有 Python MCP server 的现成范式**（`mcp-server/`，`python:3.11-slim`）
- **推荐做法**：把 Go 侧工具以 MCP 形式暴露（新增/扩展现有 mcp-server），AgentScope 通过 Toolkit 挂载 → **工具逻辑零重写**
- **不推荐**：用 Python 重写 24 个工具 —— 这是方案 B/C 里最大的一块成本
- 例外：`knowledge_search`（1,612 行）、`grep_chunks`、`database_query`、`data_analysis` 这些重工具要逐个验证参数 schema 与输出截断语义（现有截断 `internal/agent/tools/truncate.go:12`，默认 16,000 字符）

接缝④ 计费与限流

- 现状：计费**完全依赖 Go 侧从模型响应里拿到 usage**——`Chat` 用 `response.Usage`（`internal/application/service/model_billing.go:406-409`），`ChatStream` 从流末 `finalUsage` 取值（`:453-470`）；模型调用被统一包一层 `billingChat`（`internal/application/service/model.go:569`）；工具调用另有 `tool_call` 计费（`internal/application/service/tool_billing.go:14`）
- 模型并发闸门是 Redis 分布式信号量（`internal/models/limiter/limiter.go:117`），限流见 `internal/ratelimit/limiter.go:66`
- **风险**：若 AgentScope **直连 LLM**，Go 侧拿不到 usage → **token 计费静默丢失**，且并发闸门失效
- **必须做**：走 Go 的 OpenAI 兼容代理（接缝②的 (b)），让计费与闸门继续在 Go 侧生效；或新增 usage 回传结算入口并做影子对账

3.2 方案 A：旁路试点（推荐先做）

定位：AgentScope 独立进程，只服务**新增场景**（多 Agent 编排类，如一个"多渠道招生策略编排"或"专家团队协作"）。现有 `/agent-chat` 主链路**一行不动**。

#	工作项	说明	人日
1	服务骨架	`create_app` 或纯库 + FastAPI；含会话存储（Redis）与 workspace	3
2	模型凭证桥	Go 侧新增 OpenAI 兼容代理端点（复用 `billingChat`），Python 侧只配 Go 的 base_url	3
3	工具复用	扩展 `mcp-server/` 暴露所需 Go 工具（MCP SSE / Streamable HTTP）	5
4	事件翻译	AgentScope 事件 → 本项目 `response_type` 子集（含 `agent_steps` 回写）	4
5	计费打点	走代理则自动生效，需验证 token 与工具计费不丢	2
6	部署	Dockerfile（Python 3.11）+ compose 服务 + helm + 健康检查	2
7	联调/回归/压测	含租户隔离验证、断线重连、灰度开关	5
	合计		**≈ 24**

收益：拿到真多 Agent 编排 + 沙箱隔离 + 长期记忆；风险被隔离在新场景，不碰现网主链路。 **成本特征**：几乎全是新增代码，**存量零改**（除新增代理端点）。

3.3 方案 B：引擎替换（保留 Go 平台层）

定位：用 AgentScope 替换 `internal/agent/`，Go 退化为网关 + 平台能力（鉴权、租户、RAG、计费、IM、Admin）。

#	工作项	说明	人日
0	方案 A 全部	作为子集	24
1	24 个工具全量打通	逐个对齐参数 schema、输出截断、审计；重工具（`knowledge_search` 等）单测重建	15
2	事件协议全量对齐	16 种 `ResponseType` + 审批/HITL + MCP OAuth + 中断恢复	12
3	上下文策略替换	现有 compaction 语义（reserve 16k / keep 20k）与 AgentScope 原生压缩对齐 + 回归	6
4	会话历史迁移	`LoadAgentHistory` 的 `agent_steps` 展平逻辑迁移/兼容（`session_agent_qa.go:139`）	6
5	计费与限流改造	保证 token / 工具计费零丢失 + 影子对账 + 并发闸门	8
6	前端契约微调	thinking / tool_call / 引用 / 审批的渲染差异	6
7	两套契约归并	`/agent-chat` 与 `/agent-runs`（专家工作流或改用 Agent Team）	10
8	灰度与回滚	双跑对比、开关、运维手册	8
9	观测性打通	Langfuse ⟷ OpenTelemetry 双写或迁移	4
	合计		**≈ 99**

排期参考：≈ 5 人月。3 人并行约 **7~8 周**（不含风险缓冲）；单人约 20 周。

被"废弃"的存量：19,653 行 Go Agent 代码（其中工具 12,320 行若走 MCP 可救回大半），以及 `internal/agent/compaction/`、`internal/agent/approval/`、`internal/agent/skills/` 的对应能力。

3.4 方案 C：平台级采用 Agent Service（不建议）

定位：直接以 AgentScope Agent Service 承压，用它的多租户/会话/RAG/渠道/调度。

#	工作项	人日
0	方案 B 全部	99
1	鉴权与租户体系替换	用本项目 Auth/RBAC/API Key（`internal/middleware/auth.go:124`、`rbac.go:67`、`api_key_gate.go:107`）替换 `X-User-ID` 占位；tenant 模型对齐
2	与现有模块去重/取舍	IM 渠道（`internal/im/` 9 个）、RAG（`chat_pipeline/` 6,638 行）、组织与资源分享、调度——**要么砍本项目、要么砍框架**
3	计费体系重建	AgentScope 无计费，需在新链路重建
4	数据与契约迁移	会话/消息/Agent 配置迁移 + Web/Admin 前端适配或重写
5	运维改造	新增 Redis 依赖、多进程、消息总线、分布式（官方 WIP）
	合计	**≈ 190~215**

不推荐理由：① 约 10 人月起，且伴随**产品能力取舍**（要决定砍掉自研 IM/RAG 还是不用框架的）；② 框架侧分布式部署仍是 WIP，把核心链路押上去等于替上游做稳定性验证；③ 本项目最值钱的资产（租户/计费/Admin 后台/行业模板）**一样都用不上框架的**。

3.5 隐性成本（三条，容易被低估）

1. **双语言 Agent 栈**：只要工具、提示词、上下文策略在 Go 与 Python 各存一份，改一个业务规则要动两处。方案 A 用 MCP 复用工具正是为了压住这条。
2. **可观测性割裂**：项目用 Langfuse span，AgentScope 用 OpenTelemetry，两条链路对不齐会让排障成本翻倍。
3. **测试资产重估**：现有测试大量挂在 `Execute` 与工具接口上，换引擎等于这批回归基线要重做。

3.6 「只使用 AgentScope 多 Agent 引擎」的精确成本

3.6.1 先纠正一个认知：2.0 里"多 Agent 引擎"不是纯库

AgentScope 2.0 的三种协作模式共用同一 `reply_stream()` 契约，但**依赖层级不同**：

模式	代码位置	谁决定协作	拓扑	依赖服务层?
Agent Team（Leader–Worker）	`agentscope/app/_tool/`、`app/middleware/`	**LLM**（leader 调团队工具）	星型 + 异步消息 + 唤醒	**是**（MessageBus / Redis / wakeup）
Pipeline	`agentscope/pipeline/`	**开发者代码**	固定流程	否（纯库）
A2A	`agent/_a2a_agent.py`	外部系统	点对点	否（纯库）

- Team 工具共 5 个：`TeamCreate` / `AgentCreate` / `AgentInvite` / `TeamSay` / `TeamDelete`；成员是**独立会话**，`TeamSay` 把消息作为 HintBlock 写入对方收件箱（MessageBus）并发 wakeup，对方空闲时由 `_wakeup_dispatcher.py` 拉起——**这套机制长在服务层上，不是库能力**。
- 纯库可用的多 Agent 原语是：**Pipeline**（顺序/并行/固定拓扑）、**`GoalPipeline(executor, verifier)`**（执行者 + 验收者收敛循环）、**MsgHub**（进程内发布订阅广播）、**agent-as-tool / subagent**（把 Agent 包成可调用工具做层级委派）。

> **结论**：「只使用多 Agent 引擎」= 用 Pipeline / MsgHub / agent-as-tool **自己写编排**，能把现在写死的六段流水线升级为可组合、可并行、可收敛（executor-verifier）的编排；**但拿不到「leader 运行时动态派生 worker」**——那部分必须连服务层（Redis 消息总线 + 唤醒调度）一起引入。**这是最容易踩的预期落差。**

3.6.2 四档成本

档	形态	拿到的多 Agent 能力	人日
0 · PoC 验证	纯库，2 个 Agent + 1 个 MCP 工具 + 事件回传；不落库、不硬化、不上 compose	只验证表达力	**5~7**
1 · 纯库编排	Pipeline / MsgHub / agent-as-tool，旁路新场景，不碰 `agent_runs`	固定拓扑 + 并行 + verifier 收敛	**≈ 24**
2 · 含 Team	档 1 + app 层（Team 工具 + Redis 消息总线 + 唤醒调度）	Leader 动态派生 worker、异步汇报	**≈ 32~34**
3 · 替换专家工作流	档 2 + 兼容 `agent_runs` 三表（异步 / 取消 / 重试 / 版本链）	同档 2，但替掉存量	**≈ 42**

3.6.3 档 1（≈24 人日）工作项明细

#	工作项	说明	人日
1	Python 最小服务骨架	只装 `agentscope` 库 + FastAPI 单端点；**不引入** app 层、WebUI、自带 RAG、自带渠道、自带调度	2
2	模型走 Go 代理	Go 新增 OpenAI 兼容端点（复用 `billingChat`，`internal/application/service/model.go:569`）；Python 侧 `Credential(base_url=Go)`	3
3	工具经 MCP 复用	只暴露该场景需要的 5~8 个工具（非全部 24 个）	4
4	**编排代码**	角色定义 + 提示词 + 工具权限矩阵 + Pipeline / agent-as-tool 组合	5
5	事件翻译	AgentScope 事件 → `response_type` 子集（对照 `internal/types/chat.go:91-126`）	4
6	部署	Python 3.11 镜像 + compose 服务 + helm + Redis key 隔离	2
7	联调 / 回归 / 压测 / 租户隔离验证	—	4
	合计		**≈ 24**

一个可省的变体：现有专家工作流本身就是**异步**的（`agent_runs` + `agent_run_events`，`migrations/versioned/000099` / `000103`），不共用 `/agent-chat` 的实时 SSE 契约。若把 AgentScope 只用在异步报告场景，事件翻译退化为"落库映射"（无实时中断 / HITL），可压到 **≈ 20 人日**。

3.6.4 档 2 的增量（+8~10 人日）

#	工作项	人日
1	app 层装配：`RedisStorage` + `RedisMessageBus` + workspace manager，**新增一项基础设施依赖**	3
2	Team 工具与角色投射（`SubAgentTemplate` 定义只读 explorer 等角色、权限模式锁死）	3
3	汇报/催促/唤醒语义对齐（`TeamMemberLoopMiddleware` 未汇报最多催 3 次、`InboxMiddleware` 下一轮 reasoning 前注入）	2
4	HITL 投射对齐（`_subagent_hitl.py` 把 worker 的确认请求投到 leader → 映射到项目审批事件）	2

3.6.5 档 3 的增量（+8 人日）

#	工作项	说明	人日
1	三表对接	`agent_runs` / `agent_run_events` / `agent_run_steps` 落库与序号分配	4
2	语义对齐	取消 / 重新生成 / 版本链（`POST .../cancel`、`regenerate`；`frontend/src/api/agent-run.ts:31-99`）	4

3.6.6 决策规则（按目标选档）

你的目标	选档	理由
想更快产出**更多固定形态**的专家流程	**档 1**	≈24 人日，**不新增 Redis 依赖**，纯库、爆炸半径小
要一个**通用编排器**，让模型自己决定派几个角色	**档 2**	必须连服务层；要接受新增 Redis 消息总线 + 官方分布式仍 WIP
已确定要**下线**现有专家工作流	**档 3**	否则 1,837 行的 `agent_run_expert_workflow.go` 与 Python 编排**双份并存**

必须说清的一句：档 1 的"多 Agent"本质仍是**代码写死的拓扑**——与现在的 `agent_run_expert_workflow.go:140` 属同一类东西，只是表达力更强（并行、verifier 收敛循环、agent-as-tool）。**若收益预期是"动态分工"，档 1 会让人失望，必须直接上档 2。**

3.7 「只使用智能体本体」（不用"智能体即服务"）的成本

3.7.1 口径

官方文档「智能体即服务」一节 = **Agent Service 整层**（智能体团队 / 标准操作流程服务 / 资源共享 / RAG 服务 / 工作区管理 / MCP 与技能中心 / 消息渠道 / API）。「只使用智能体」= **只取 `ReActAgent` 单智能体库本体**，该节全部不引入。

3.7.2 结论先行：这是性价比最差的一档

维度	多 Agent 引擎（§3.6）	智能体本体（本节）
对项目而言的性质	**增量**（旁路新场景，主链路不动）	**替换**（必须动 `/agent-chat` 主链路）
四道接缝	只需碰与场景相关的子集	**一个都躲不掉**
单 Agent 循环本身	顺带获得	对照表结论**持平**（§2），增量 ≈ 0
与其他档的关系	—	是档 1 / 档 2 的**真子集**（Pipeline / Team 的每个成员就是 ReActAgent）

三条理由：

1. **单 Agent 循环是本项目已经做完且做得不差的部分**：自研 19,653 行，含工具并发、超时、上下文压缩、审批、输出截断；AgentScope 在这一面没有明显优势（§2 对照"持平"）。
2. **替换成本却一点不少**：替换主链路引擎 = 16 种 `ResponseType` 全量对齐 + 24 个工具全量打通 + 计费对账 + 历史迁移，一个都不能省。
3. **单独为它立项没有意义**：只要按 §3.6 档 1 或档 2 立项，单智能体能力是**免费附赠**的。

3.7.3 两种用法与成本

用法	性质	成本	建议
替换主链路引擎	替换	**≈ 80~85 人日**	❌ 不建议（增量 ≈ 0）
仅新场景用单智能体	增量	12~15 人日	⚠️ 不值得单独立项——被 §3.6 档 1 / 档 2 包含

3.7.4 「替换主链路」明细（≈82 人日）

#	工作项	说明	人日
1	Python 服务骨架	只装 `agentscope` 库，无 app 层	2
2	凭证桥	Go 新增 OpenAI 兼容代理（`model.go:569`），凭证不出 Go	3
3	24 个工具全量 MCP 化	逐个对齐 schema / 截断（`truncate.go:12`）/ 审计	15
4	事件协议全量对齐	16 种 `ResponseType` + `agent_steps` 回写 + 审批 + MCP OAuth + 中断	12
5	上下文压缩对齐	reserve 16k / keep 20k 语义（`internal/agent/compaction/`）与原生压缩	6
6	会话历史迁移	`LoadAgentHistory` 的 `agent_steps` 展平（`session_agent_qa.go:139`）	6
7	计费与限流	usage 不丢 + 影子对账 + 并发闸门（`models/limiter`）	8
8	前端契约微调	thinking / tool_call / 引用 / 审批渲染差异	6
9	**skills / approval / sandbox 迁移**	`internal/agent/skills/`（1,100 行）+ `approval/`（682 行）映射到 AgentScope 的 workspace 与 permission 体系	8
10	灰度 / 双跑 / 回滚	开关 + 对比 + 运维手册	8
11	观测性	Langfuse ⟷ OpenTelemetry	4
12	联调 / 回归	—	4
	合计		**≈ 82**

> 对比方案 B（≈99）：本档省掉的是**多 Agent 归并**（两套契约、Agent Team，共 17 人日）；其余全部照付。

3.7.5 什么时候这一档才成立

只有当**自研引擎撞到明确天花板**时才值得付这笔替换成本，例如：需要 AgentScope 独有的实时中断恢复、沙箱生态（E2B / K8s）、长期记忆中间件（ReMe / Mem0）、`ModelRouterMiddleware` 模型路由等成为刚需。**按 §2 的对照表，目前看不到这个天花板——所以不建议为"智能体本体"单独立项。** 要 AgentScope 的价值，走 §3.6 的多 Agent 引擎旁路；主链路保持自研。

3.8 投产比测算：换这个框架能不能打正

3.8.1 前提与口径

- 人日单价假设 **¥2,000**（含社保、设备与管理分摊），敏感度用 **¥2,500**
- 只算研发投入，不含商务、运营与机房
- **收益侧只承认可折算为毛利的增量**；与框架重叠的能力（多租户、RAG、IM、计费）**不产生增量收益**，不计入

3.8.2 成本侧

方案	一次性（人日）	¥2,000/人日	¥2,500/人日	永久性成本
旁路多 Agent（§3.6 档 1）	24	¥4.8 万	¥6.0 万	双栈（工具走 MCP，可控）
旁路含 Team（§3.6 档 2）	34	¥6.8 万	¥8.5 万	＋新增 Redis 消息总线运维
只用智能体本体（§3.7）	82	¥16.4 万	¥20.5 万	双栈全量 ＋废掉引擎本体
整引擎替换（方案 B）	99	**¥19.8 万**	**¥24.8 万**	双栈 ＋多 Agent 归并
平台级采用（方案 C）	190~215	**¥38~43 万**	**¥47.5~53.8 万**	平台重建 ＋上游 WIP 风险

3.8.3 隐性成本（不进人日表，但必须计入）

#	项	量级
1	**沉没资产作废**	走 MCP 复用工具 → 只废引擎本体（19,653 − 12,320 = 7,333 行 ≈ 24 人日 ≈ **¥5~6 万**）；若 Python 重写工具 → 废 19,653 行 ≈ 65 人日 ≈ **¥13~16 万**
2	**永久双栈维护**	按 Agent 相关研发年投入 480 人日、同步损耗 10% 计 = 48 人日/年 ≈ **¥9.6~12 万/年**——只要两套并存就一直在扣
3	**上游风险**	官方标注分布式部署 WIP，近月高频发版（SOP / TeamPipeline / Realtime voice），返工概率不低

> 第 1 条是关键：**换框架的隐含代价是让已投入资产归零**——其账面价值与替换成本本身是同一量级。

3.8.4 收益侧：三条增量，逐条判断能不能变现

增量	是否必须换框架	可折算收益	判断
动态多 Agent 分工 → 报告质量	**必须换核心引擎**	客单价 / 续费	⚠️ **最大不确定性，需 A/B 证明**
沙箱隔离（E2B / K8s）解锁代码执行类场景	**不需要**（可独立接沙箱服务）	新场景收入	独立可采
长期记忆（ReMe / Mem0）→ 留存	**不需要**（两者都是独立组件）	留存 / 复购	独立可采
新场景交付效率	不需要（§3.6 旁路已覆盖）	研发提效	旁路已覆盖

决定性的一条：三条增量里**只有第一条必须换框架**，另外两条不换框架也能拿到——而第一条恰恰是最不确定的。

3.8.5 回本门槛

方案	3 年总投入	打正所需年新增毛利	折算业务（单园所年费 1 万）
旁路多 Agent（档 1）	¥4.8~6 万	**≥ ¥1.6~2 万/年**	多卖 **2 个**园所
旁路含 Team（档 2）	¥6.8~8.5 万	≥ ¥2.3~2.8 万/年	多卖 3 个园所
整引擎替换（方案 B）	**¥50~61 万**（含双栈 3 年）	**≥ ¥17~20 万/年**	多卖 **17~20 个**园所
平台级采用（方案 C）	¥67~90 万	≥ ¥22~30 万/年（且要放弃部分自研能力）	多卖 22~30 个园所

3.8.6 结论

1. **换整框架（方案 C）：投产比打不正。** 花的是平台重做的钱（¥38~54 万一次性 + 平台重建风险），买的是三处增量，其中两处不需要换框架，第三处未被证明。
2. **换引擎（方案 B）：能否打正取决于一个尚未验证的假设。** 需每年新增毛利 ≥ ¥17~20 万才回本；而"动态多 Agent 分工带来业务增益"这一假设**尚未被验证**。假设成立就打正，不成立就是净亏。
3. **旁路多 Agent（§3.6 档 1/2）：投产比最容易打正。** 成本约方案 B 的 1/4，**无永久双栈依赖**，回本门槛仅 ¥1.6~2.8 万/年。

3.8.7 建议动作（Go / Kill 判据）

步	动作	花费	判据
1	做 §3.6 档 0 的 PoC，量化"动态多 Agent 分工 vs 现有固定流水线"的质量差	5~7 人日 ≈ **¥1.2~1.8 万**	—
2a	若提升 **< 20%** 或无法量化 → **判 Kill**	—	换引擎的投产比必为负，停在旁路档 1
2b	若提升显著且能挂到明确产品收益假设 → 按 **档 2** 立项	—	不必走整引擎替换（省 ¥13~20 万 + 双栈）

> 一句话：**用 ¥1.5 万买一个答案，再决定要不要花 ¥50 万。** 这是本报告唯一推荐的决策顺序。

3.9 价值判断：值得引入的是能力，不是框架

3.9.1 先把「价值」拆成三层

层	回答的问题	归口
能力价值	它给你什么你现在没有的	技术（本节 3.9.2）
经济价值	这些能力能不能换成钱	商业（§3.8 已算）
战略价值	押注它对不对	组织与演进（3.9.4）

三层里**能力价值是根**——只有真增量才谈得上后两层。所以先做一次逐项评级。

3.9.2 能力价值：八项逐条评（真增量 / 伪增量 / 负增量）

AgentScope 能力	评级	依据
动态多 Agent 编排（Leader-Worker / TeamPipeline / SOP / A2A）	✅ **真增量，且强绑定框架**	本项目现为**硬编码五阶段流水线**（`agent_run_expert_workflow.go:140`），各步 `AllowedTools: []string{}` 纯文本生成。这是**唯一"必须换框架才拿得到"**的增量
隔离沙箱（E2B / K8s workspace）	✅ 真增量，但**不绑定框架**	本项目 `internal/sandbox` 默认 disabled（`agent_service.go:359-398`），但沙箱可作为独立服务接入，与是否用 AgentScope 无关
长期记忆（ReMe / Mem0）	✅ 真增量，但**不绑定框架**	两者本身就是独立后端组件，挂自研引擎同样可用
单 Agent ReAct 循环	⚪ **伪增量（重叠）**	自研 19,653 行，工具并发 / 超时 / 上下文压缩 / 审批 / 输出截断齐备，§2 对照结论为「**持平**」
工具系统 / MCP	⚪ **伪增量**	24 个工具 12,320 行 + 项目已有 `mcp-server/` 在跑
流式事件协议	⚪ **伪增量**	16 种 `ResponseType` 已是前端契约（`internal/types/chat.go:91-126`），比框架事件更完整
多租户 / RAG / IM / 计费 / RBAC	❌ **负增量**	引入即要求取舍或重做，是本项目护城河（§2、§0.3）
双语言 Agent 栈	❌ **负增量**	工具 / 事件 / 提示词 / 上下文策略永久双份维护（§3.8.3）

一句话：八项里**只有一项"真增量且强绑定框架"——可编排多 Agent**；其余要么能白拿（沙箱、记忆），要么是负债（重叠、双栈）。

3.9.3 经济价值：已算，结论是「旁路为正、替换为负」

按 §3.8：换整框架（方案 C）三年 ¥67~90 万，需年新增毛利 ≥ ¥22~30 万，**打不正**；换引擎（方案 B）三年 ¥50~61 万，需 ≥ ¥17~20 万/年（**折算多卖 17~20 个园所**），是否打正取决于一个**尚未验证**的假设；旁路多 Agent（档 1/2）三年 ¥5~8.5 万，需 ≥ ¥1.6~2.8 万/年（**多卖 2~3 个园所**），**最容易打正**。

3.9.4 战略价值：一个常被忽略的替代路径

反直觉但重要的一条：**Apache-2.0 开源框架的直接价值是「可读」，不是「可依赖」。**

1. **多 Agent 的难点不在运行时，在编排范式**——何时派角色、如何收敛、失败如何回退，这些 AgentScope 已写在文档与源码里。**读它、抄范式、在自研引擎上实现**，成本远低于引入一整套 Python 运行时 + Redis 总线 + 双栈。

2. **押注上游有反作用**：官方自述 Agent Service **分布式部署为 WIP**，2.0 仍高频迭代（近月新增 SOP / TeamPipeline / Realtime voice）。**把核心能力交给上游 = 让自家产品节奏跟着别人的 changelog 走。**
3. **唯一不可替代的战略项是生态与人才**（招得到会用的人、社区有现成范式），但本项目是 Go 栈，**这项收益要打折。**

3.9.5 结论

1. **「AgentScope 有没有价值」——有，但价值不在框架，在三项能力。** 其中只有一项（可编排多 Agent）与框架强绑定。
2. **对你们的处境：作为「设计参考 + 旁路引擎」有价值；作为「平台底座 / 主引擎」价值为负。**
3. **「更火」不改变结论。** 框架流行度影响的是人才与文档供给，**不影响你们与它的能力重叠度**——重叠的部分不会因为框架更火就变得不重叠。
4. 因此 §3.8 的 Go / Kill 判据不变：**唯一前置动作仍是那个 5~7 人日的 PoC**，它同时回答"多 Agent 强多少"与"值不值得引入这个框架"。

3.9.6 价值成立的三个前置条件（全部为真才立项）

#	条件	不成立时的动作
1	动态多 Agent 分工对产出质量的**真提升 ≥ 20%**	提升不显著 → **不引入**，顶多在自研引擎上抄范式
2	存在真实的**隔离执行场景**（代码执行 / 数据处理）	无场景 → 沙箱不是理由，别为它引入
3	长期记忆必须解决，**且不能用独立组件外挂**（ReMe / Mem0）	能外挂 → 不换框架

3.10 反向问：不引入框架，在现有工程上自研复刻这些能力的复杂度

这是前面所有问题的镜像。前面算的是「把 AgentScope 接进来要多少」，这里算的是「**不接它，自己把这些能力造出来要多少**」。

3.10.1 先算「已有多少」——可复用底子清单

已有底子	位置	对自研复刻的意义
ReAct 引擎与单步循环	`internal/agent/engine.go:205` / `:361` / `:467`	**每个 Agent 成员的运行时白送**，不用重写
「流程内多次调用引擎」的现成先例	`agent_run_expert_workflow.go:578` / `:593`（各自 `NewAgentEngine` + `Execute`）	**agent-as-tool / 固定 Pipeline 已有可抄的模式**
异步运行基础设施（含回放/重试/版本链）	`agent_runs` / `agent_run_events` / `agent_run_steps` / `agent_run_threads`（迁移 `000099` / `000103` / `000104`）	多 Agent 编排的**持久化与事件回放表已存在**
Redis 与流式管理器	`internal/stream/redis_manager.go` + `go-redis/v9`（`go.mod:50`）	消息总线 / 唤醒调度**无需新增中间件**
沙箱（**已实现、未启用**）	`internal/sandbox` 1,514 行（`docker.go` / `local.go` / `manager.go` / `validator.go`），默认 disabled（`agent_service.go:387-396`）	隔离执行是**「上架」而非「从零造」**
Skills 体系	`internal/agent/skills` 1,100 行	与 AgentScope skills 概念已对齐
工具系统 + MCP	24 工具 12,320 行 + `mcp-server/`	工具层**完全不用动**
审批 / HITL	`internal/agent/approval` 682 行	Permission & HITL 已有雏形
上下文压缩	`compaction/` + `token/` 1,358 行	自动压缩已有；缺的是「中间件管线」抽象

结论：AgentScope 的能力面里约七成你们已有等效物。 真正要自研的只有**编排层、调度层，加上几个中间件**。

3.10.2 逐能力复杂度分级（自研口径）

#	能力	现有底子	自研要补什么	复杂度
1	固定拓扑 Pipeline（顺序/并行/归并）	串行调多引擎有先例；并行已有 `errgroup`（`act.go:190`）	分支 + 归并 + 成员参数化	低
2	agent-as-tool（层级委派）	工具可注入 service、引擎可实例化	一个 delegate 工具 + **递归深度 / 预算 / 防环**	低
3	GoalPipeline（executor + verifier 收敛环）	无	收敛判据 + 轮次上限 + 不收敛的升级策略	中
4	MsgHub / 共享上下文广播	**引擎跨轮无状态，历史每轮从 DB 重建**（`engine.go:31-35`）	会话级共享消息层 + 成员可见范围	中
5	**动态 Leader-Worker（Team）**	无（专家流是硬编码五阶段）	工具化的成员增删/发言 + 收件箱注入 + **唤醒调度** + 生命周期并发	**高**
6	A2A 协议（跨系统）	无	协议 + 鉴权 + 发现（可延后）	高
7	**实时中断 / 恢复**	**无**——现有引擎没有可中断点	主循环加中断点 + 状态快照 + 恢复重放	**很高**
8	沙箱隔离后端（E2B / K8s）	`internal/sandbox` 已实现 docker/local，默认关	启用 + 新增后端 + 资源配额	中
9	长期记忆（ReMe / Mem0 式）	有 organize 记忆模型与向量检索	记忆抽取 / 写入 / 召回策略 + 与检索打通	中高
10	中间件管线（工具结果卸载 / 注入）	compaction 已有	管线抽象	低~中
11	Permission & HITL 细粒度化	approval 682 行	补齐授权粒度	低
12	观测性（OTel span 语义）	Langfuse 已在用	补 span 语义	低

3.10.3 三个自研包的人日（同 §3.8 口径）

自研包	内容	人日	对标引入档	差值
包 A 固定编排	引擎成员化 5 · Pipeline 6 · agent-as-tool 4 · verifier 收敛 5 · 编排落库回放 5 · 事件与前端 6 · 联调 6	**32~40**	AgentScope 档 1 ≈ 24	自研贵 8~16
包 B 动态多 Agent	包 A + 共享上下文 7 · 收件箱注入 5 · **唤醒调度 10** · 成员生命周期 7 · 熔断与可观测 8 · 多 Agent 回归 8	**70~88**	AgentScope 档 2 ≈ 34	自研贵 **36~54**
包 C 高级能力	包 B + **实时中断恢复 30~45** · A2A 15~20 · 沙箱后端 10~15 · 长期记忆 20~30	**145~200**	Agent Service 整层	—

3.10.4 三条判断

1. **纯人日口径，自研不便宜。** 包 A 比引入档 1 贵约 50%，包 B 贵一倍以上——**能力越往上（编排、调度），自研的边际成本越高**，因为这部分没有可复用的骨架，且失败模式（发散、死循环、角色越权）要自己踩一遍。
2. **但自研买到的是框架的「负增量」。** 零 Python 运行时、零新增中间件（Redis 已在）、零双栈、与本项目租户 / 计费 / 16 种 `ResponseType` 事件体系**天然同构**。所以取舍不是「贵不贵」，而是「**要不要为了省 36~54 人日，永久多维护一个 Python 栈**」。
3. **复杂度分布有明确规律：越贴近已有积累的越划算，越依赖框架运行时的越不划算。**

能力类型	代表	自研划算度
中间件类（贴近已有）	沙箱、长期记忆、压缩、HITL、观测	✅ **自研更划算**——已有底子，且本就不绑定框架（§3.9 左上象限）
编排类	Pipeline、agent-as-tool、verifier 收敛	⚠️ 五五开——人日接近，取舍看要不要 Python 栈
调度与引擎级	动态 Team、**实时中断恢复**	❌ **引入更划算**——从零造最难，且官方也在打磨

3.10.5 唯一的结构性缺口

实时中断 / 恢复（#7）是自研路径上唯一会「打破旁路前提」的项目。

现有引擎**跨轮无状态、历史每轮从 DB 重建**（`engine.go:31-35`），没有任何中断点。要支持运行中打断并恢复，必须改主循环契约——而主循环事件要经 `agent_stream_handler.go:100-114` 映射成 **16 种 `ResponseType`**，向前端扩散。也就是说：**包 A、包 B 都可以做成旁路、主链路零影响；只有包 C 里的中断恢复会动主链路**，等于回头付 §3.7 那 82 人日的一部分。

反过来说：**只要「实时中断恢复」不是产品刚需（按现状看不是），自研路径整体不碰主链路，复杂度是可控的。**

> **由本节得出的最优解是混合而非单选**：中间件类自建（省钱且不绑定），编排调度类引入（省 36~54 人日且避免重造最难的部分）——这正是 §3.9「值得引入的是能力，不是框架」的工程落法。

3.11 第三条路：上游框架（WeKnora）自己有没有这些能力

前面所有分析都在「引入 AgentScope」与「自研复刻」之间二选一。但本项目是 **WeKnora 的定制分支**，所以存在第三条路：**上游自己有没有这些能力？** 结论是会改写前面几节的判断。

3.11.1 事实基线：睿乐是 WeKnora 定制分支，落后两个 minor

项	睿乐当前	上游 WeKnora
版本	`VERSION` = 0.7.0	**0.8.2**（2026-09-24），main 已推进到 10-09
Go 模块路径	`github.com/Tencent/WeKnora`（未改）	同
v0.8 新增的三个包	**全都没有**：`internal/ipclass`、`internal/application/service/memory`、`third_party/anydoc-go`	有
`internal/sandbox`	**旧版形态**：`local.go` / `local_unix.go` / `local_windows.go` + `docker.go`	v0.8.0 **已移除 Local 后端**，重写为 `RemoteSandboxClient`（Docker Engine API / E2B / Cube）
迁移号 `000080~000090`	已被睿乐自研语义占用（`tenant_invitation` / `org_members` / `personal_org_kb_v2` / 计费 / `ruile_prompt_identity` …）	官方是**另一套语义**（自动标签 / 消息产物 / 沙箱 / 记忆 / Skill Catalog）→ 同号不同义

两条推论：

1. **睿乐的 Agent 沙箱与部分 Agent 能力源自上游 0.7.0 基线，不是从零自研**——这修正 §1.1 的表述口径：准确说法是「基于上游 WeKnora 的 Agent 模块深度定制」，自研部分主要在业务层（整理 / 服务 / 专家包 / Agent Run）。
2. **0.8.0 是一次集中发布**，主题恰好是「Skill、沙箱、记忆和解析能力」（本机 `docs/WeKnora-v0.8.0与睿乐当前版本重要功能对比.md:10`）。

3.11.2 逐项核查：AgentScope 的增量，上游有没有

#	能力	AgentScope	上游 WeKnora	睿乐当前	判断
1	**多 Agent 编排**（Leader-Worker / Team / A2A）	✅ 强绑定框架	❌ **没有**（全版本 changelog 无 multi-agent / subagent / agent team / orchestration）	❌ 只有硬编码流水线	**两边都没有 → 这才是真缺口**
2	沙箱隔离（session 级持久化，shell / 文件 / 产物收集）	✅	✅ **v0.8.0**（迁移 `000082_tenant_sandbox_config` / `000083_session_sandbox_config`；Docker · **E2B** · Cube）	⚠️ 旧版 local/docker	上游有新版 → **移植**
3	跨会话长期记忆	✅ （ReMe / Mem0）	✅ **v0.8.0**（迁移 `000084_memory`；profile / preference / fact / task / interest，用户确认后生效，`search_memory`，v0.8.2 再修排序）	❌ 无（只有 `organize_memories`，模型不同）	上游有 → **移植**
4	租户级 Skill Catalog + 安装闭环	⚠️ Skill Hub	✅ **v0.8.0**（`000086` ~ `000090`；ClawHub / SkillHub / GitHub·GitLab / zip，环境变量 `000089`）	⚠️ 有执行、无目录与安装闭环	上游有 → **移植**
5	上下文压缩 + token 用量归属 + 提示词缓存标记	✅	✅ **v0.8.0**（`000085_message_usage`）	⚠️ 有压缩、无 per-turn 用量归属	上游有 → **差异移植，勿重复建**
6	消息产物库 + 时间线	⚠️	✅ **v0.8.0**（`000081_message_artifacts`）+ v0.8.2 artifacts library	⚠️ 有 Agent Run 产物	上游有
7	会话控制（运行中追加需求 / 对话分叉 / rewind 并把沙箱回到检查点）	⚠️	✅ **v0.8.2**	❌ 无	上游有
8	内置 MCP Server（Go）	⚠️	✅ **v0.8.2**（Python 版已标记弃用）	⚠️ Python 版 `mcp-server/`	上游有
9	本地浏览器（BrowserSkill，可点击 / 填表 / 任务窗）	⚠️	✅ **v0.8.2**	❌ 只有 chromedp 取 HTML	上游有
10	进程内 Office 解析（anydoc，Go↔Rust cgo）	—	✅ **v0.8.0**	❌ 无	上游有
11	多模态生成（图 / 视频 / 3D）	⚠️	❌ 无	❌ 无	都没有

3.11.3 三条判断（本节最重要）

1. **前面几轮当作「AgentScope 的真增量」的两项——沙箱与长期记忆——上游框架都已经有**，而且是 0.8.0 的头条功能。再加上 Skill Catalog、产物库、token 用量归属、会话控制、本地浏览器，**上游一次 release 就覆盖了 AgentScope 增量里除多 Agent 之外的全部。**
2. **AgentScope 唯一「强绑定框架」的增量（可编排多 Agent），上游也没有。** 这让它的定位彻底清楚了：**它不是"能力的补丁"，而是一个单一用途的多 Agent 编排外挂。**
3. **更关键的是血统**：上游这些能力与本项目**同语言（Go）、同迁移体系、同租户 / RBAC 模型、同前端契约**——移植成本远低于引入 Python 栈，也比 §3.10 里的自研方案便宜一个数量级。

3.11.4 移植成本：项目自己已有方案与估算

本机 `docs/WeKnora-v0.8.0三任务迁移升级方案.md:109-119` 已给出三任务安排：

任务	内容	项目自估工作量
任务一	v0.8.0 基线与迁移工具（解决迁移链冲突、导出数据清单）	2~3 工程日
任务二	**重点功能升级与自研模块重新接入**	3~5 工程日
任务三	数据迁移、灰度与生产切换（+72 小时观察）	1~2 工程日
合计	—	**约 8~14 人日**（前后端并行 5~7 个工作日）

⚠️ **三条必须附上的限定**：

1. **这是项目自己文档里的估算，不是外部核价**，且**风险全在任务二**——它要把整理 / 服务 / 专家包 / Agent Run / 服务空间五套自研模块重新挂到新基线上；3~5 工程日属乐观值。
2. **迁移号必须重编号**：官方 `000082` 是 `session_sandbox_config`，睿乐 `000082` 是 `personal_org_kb_v2`——同号不同义，直接合并会破坏迁移历史。方案要求「每个功能先做 `CREATE TABLE/ADD COLUMN` 兼容迁移，禁止删除睿乐字段」（`WeKnora-v0.8.0与睿乐当前版本重要功能对比.md:100-105`）。
3. **长期记忆存在模型冲突**：官方 `memory`（模型抽取的用户偏好 / 事实 / 任务）与睿乐 `organize_memories`（用户主动记录与录音转写）**不是同一个模型**，需先定义边界，不能共用表（同文档 `:148-156`）。文档还明确要求**沙箱保留 feature flag、先 Skill Catalog 后沙箱**。

3.11.5 结论：三路合一（含对 §3.10 判断的修正）

能力	最优路径	理由
沙箱 / 长期记忆 / Skill Catalog / 产物库 / token 用量 / 会话控制	**移植上游 v0.8**	同血统，8~14 人日量级；比引入 Python 栈便宜，比自研（§3.10 包 C 里沙箱 10~15 + 记忆 20~30）便宜一个数量级
多 Agent 编排	**引入 AgentScope 旁路**（24 / 34 人日）**或**自研包 A/B（32~40 / 70~88）	**上游也没有**，只能二者择一
HITL / 审批 / 观测 / 事件协议（已有底子）	自研（已有）	已有等效物，无需动

> **对 §3.10 判断 3 的修正**：§3.10 说「中间件类（沙箱、长期记忆）自建更划算」——**更准确的说法是「移植上游更划算」**。上游已把这两项做完、且版本更新（新增 E2B / Cube 后端与用户确认式记忆），自建等于重做一遍。

一句话：这个问题的答案把整份报告的结论推向了**第三个选项**——既不是「引入 AgentScope」，也不是「自研复刻」，而是**「先把上游 0.8 拉回来」**。上游一次 release 覆盖了 AgentScope 增量里除多 Agent 之外的全部能力，成本 8~14 人日；剩下的多 Agent 缺口再用 AgentScope 旁路补。

AgentScope 的角色因此从「框架选型」降级为「多 Agent 编排的单一用途外挂」——这也与 §3.9、§3.10 的结论收敛到同一点：**值得引入的是能力，不是框架；而大部分能力，上游已经给了。**

---

4. 问题 2：单独部署一个应用服务是否可行

4.1 结论

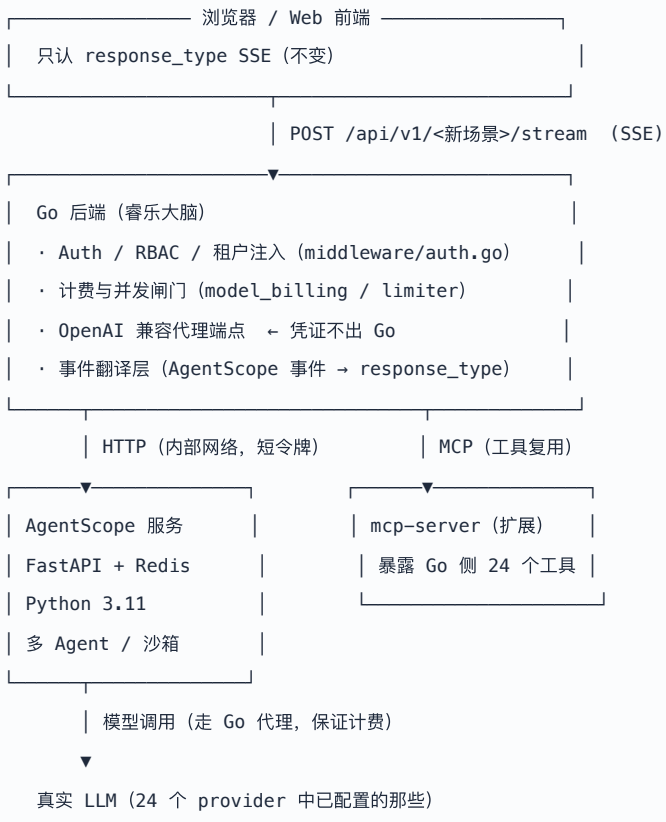
可行，而且是本项目已有既定范式。 门槛不在"能不能部署"，而在**接缝是否能对齐**（见 3.1）。

4.2 现成样板：docreader 与 mcp-server

样板	技术栈	接入方式	证据
docreader	Python 3.10.18 + gRPC	后端 gRPC 客户端 + `DOCREADER_ADDR=docreader:50051`	compose `docker-compose.yml:281-329`；客户端 `internal/infrastructure/docparser/grpc_parser.go:31-77`；proto `docreader/proto/docreader.proto:7-16`；K8s `helm/templates/docreader.yaml:1-70`
mcp-server	Python 3.11 + Starlette/uvicorn	后端作为 MCP client 调用（HTTP 传输）	`mcp-server/Dockerfile:1`；`mcp-server/requirements.txt`；compose 服务 `mcp`（`docker-compose.yml:827`，映射 `8082:8000`，`depends_on app`）

这意味着：新增一个 AgentScope 服务，**部署工作量只有 2 人日量级**——照抄 docreader 的四件套（Dockerfile + compose 服务 + helm 模板 + 健康检查）。

4.3 建议的服务形态



关键设计取舍（4 条）：

1. **AgentScope 不直连 LLM**，统一指向 Go 的 OpenAI 兼容代理 —— 一条设计决定同时解决凭证安全（接缝②）与计费（接缝④）
2. **工具走 MCP**，不复用 Go 的 service 实例也不重写 ——解决接缝③
3. **鉴权与租户留在 Go** —— AgentScope 的服务**不含内置鉴权系统**（只给 `X-User-ID` 占位头），本项目 Auth 中间件本来就要做前置
4. **事件翻译放 Go 侧** —— 前端一行不改，翻译层与契约变更一起受版本管理

4.4 必须补齐的清单（6 项）

#	事项	说明
1	**镜像基线**	项目现有 docreader 是 `python:3.10.18-bookworm`（`docker/Dockerfile.docreader`），而 AgentScope 要求 Python ≥ 3.11 → 必须单独建 3.11 镜像，不能复用 docreader 基线
2	**Redis 复用**	AgentScope Agent Service 用 Redis 做存储 + 消息总线；项目已有 Redis（`docker-compose.yml:371`），需**逻辑隔离 key 前缀**，避免与现有会话/限流数据互踩
3	**端口与网络**	内部网络 `WeKnora-network` 内 expose，**不发布到宿主**（照 docreader 的做法，`docker-compose.yml:295-296`）
4	**内部鉴权**	Go ↔ AgentScope 之间需要短令牌/双向校验，不能裸奔在内部网
5	**优雅降级**	AgentScope 不可用时新场景要能明确报错或降级，**不能拖垮主链路**（现有主链路不许受影响）
6	**观测性汇入**	AgentScope 的 OpenTelemetry 链路数据需汇入现有 Langfuse/监控，否则排障两头看

4.5 不建议的独立部署做法

- ❌ **直接把 AgentScope 的示例 `examples/agent_service` + `examples/web_ui` 上线**：它自带一整套 Web UI 与多租户占位实现，与本项目前端/后台会形成两套用户体系；
- ❌ **让 AgentScope 服务直连数据库或直连 LLM**：会绕过租户隔离与计费，产生无法对账的用量；
- ❌ **在 Python 侧重写 24 个工具**：这是把 12,320 行 Go 的工作量复制一份，且从此两份逻辑要同步维护。

---

5. 推荐路径

阶段	动作	产出	人日
P0 试点（推荐先做）	按方案 A 落地 1 个多 Agent 场景，走 Go 代理 + MCP 工具 + 独立服务	可运行的新场景 + 一份实测报告（多 Agent 到底比硬编码流水线强多少）	≈ 24
P1 决策点	用 P0 的实测数据回答两个问题：多 Agent 编排是否是产品刚需？收益是否覆盖双栈维护成本？	决策：继续扩场景（方案 A 复用）/ 升级为方案 B / 回退	—
P2 若升级	按方案 B 分模块替换引擎（先工具与事件，再主循环）	—	≈ 99（累计）

如果 P0 之后判断"不值得"，损失被限制在 24 人日与一个新场景模块，**主链路与存量代码零影响**——这正是推荐 A 而不是直接 B 的原因。

---

6. 风险清单

#	风险	等级	说明与对策
1	**计费静默缺失**	高	AgentScope 直连 LLM → Go 拿不到 usage。对策：强制走 Go 代理（方案 A 已内置）
2	**分布式能力未成熟**	中高	官方标注 Agent Service 分布式部署为 WIP；核心链路押上去等于替上游验证。对策：P0 只做旁路场景
3	**双栈维护成本失控**	中高	工具/提示词/上下文策略分叉。对策：工具只走 MCP 单一真源
4	**契约漂移**	中	前端只认 16 种 `ResponseType`，翻译层漏一种就是线上问题。对策：翻译层纳入回归基线
5	**凭证外泄面**	中	若走"下发解密凭证"路线，安全面显著变大。对策：走代理，凭证不出 Go
6	**上游版本迭代快**	中	AgentScope 2.0 仍在高频发布（近月新增 SOP / TeamPipeline / Realtime voice），API 有变动风险。对策：锁版本 + 季度评估
7	**许可证与合规**	低	Apache-2.0，商用友好；需做一次开源合规登记（项目已有 `THIRD_PARTY_NOTICES.md` 惯例）

---

7. 附录：证据索引

项目侧

- 引擎入口 `internal/agent/engine.go:205`；主循环 `:361`；单步 `:467`；无状态说明 `:31-35`
- 工具接口 `internal/types/agent.go:175`；24 个工具名 `internal/agent/tools/definitions.go:9-34`；注册 `internal/application/service/agent_service.go:445`、`:616-690`、`:699`
- 工具绑服务 `internal/application/service/agent_service.go:613-701`（`:622-630`）
- 并发工具调用 `internal/agent/act.go:190`；超时 `internal/agent/const.go:28`；输出截断 `internal/agent/tools/truncate.go:12`
- 引擎事件 `internal/event/event.go:52-57`；前端事件类型 `internal/types/chat.go:91-126`
- 事件翻译 `internal/handler/session/agent_stream_handler.go:100-114`；SSE 推送 `internal/handler/session/stream.go:321`
- 契约路由 `internal/router/router.go:790`（agent-chat）、`:793-797`（knowledge-search）；第二套契约 `frontend/src/api/agent-run.ts:31-99`
- 历史重建 `internal/application/service/session_agent_qa.go:139`；`agent_steps` 持久化 `internal/types/message.go:220`
- 专家工作流 `internal/application/service/agent_run_expert_workflow.go:140`、`:572`、`:578`
- 模型表 `internal/types/model.go:110-144`；凭证加密 `:148-193`；平台模型 `:37`；provider 枚举 `internal/models/provider/provider.go:15-66`
- 计费 `internal/application/service/model.go:569`、`model_billing.go:406-409`、`:453-470`；工具计费 `internal/application/service/tool_billing.go:14`
- 限流 `internal/models/limiter/limiter.go:117`、`internal/ratelimit/limiter.go:66`
- 多租户 `internal/middleware/auth.go:124`、`:251-267`；RBAC `internal/middleware/rbac.go:67`；API Key 网关 `internal/middleware/api_key_gate.go:107`
- MCP 客户端 `internal/mcp/client.go:148-217`（stdio 拒绝 `:215-217`）；MCP 服务端 Python 版 `mcp-server/`
- 沙箱 `internal/sandbox/sandbox.go:1-57`；开关 `internal/application/service/agent_service.go:359-398`

- 部署样板 `docker-compose.yml:281-329`（docreader）、`:827`（mcp）、`:371`（redis）；`internal/infrastructure/docparser/grpc_parser.go:31-77`；`helm/templates/docreader.yaml:1-70`；`docker/Dockerfile.docreader`

AgentScope 侧

- 官方仓库 `github.com/agentscope-ai/agentscope`（2.0 README：能力矩阵、Agent Service 清单、Apache-2.0、Python 3.11+）
- 部署文档 `docs.agentscope.io/versions/2.0.2/en/deploy/agent-service`（多租户 by construction、`X-User-ID` 占位鉴权、RedisStorage/RedisMessageBus、分布式 WIP、会话重放、AG-UI/A2A 适配）
- `github.com/agentscope-ai/agentscope-runtime`（已归档，能力并入 2.0）
- 模型接入（Credential + Model 解耦、任意 OpenAI 兼容 base_url）：阿里云开发者社区《AgentScope 2.0 实战指南 03 – 接入 AI 模型》

本报告所有项目侧结论均来自对当前工作区代码的只读勘察；AgentScope 侧结论来自其官方仓库与文档（截至 2026-10-09）。成本估算基于「1 名熟悉 Go 与 Python 的工程师」口径，实际排期需叠加团队并行度与风险缓冲。